



**AMITY
UNIVERSITY**

AMITY UNIVERSITY BENGALURU

NAME : P SRISHANTH

SEN NO : A866175124047

**SUBJECT : OPERATING SYSTEM
WITH LINUX INTERNALS**

LAB SLOT : L35+L36

FACULTY : DR.RAJAT BHARADWAJ

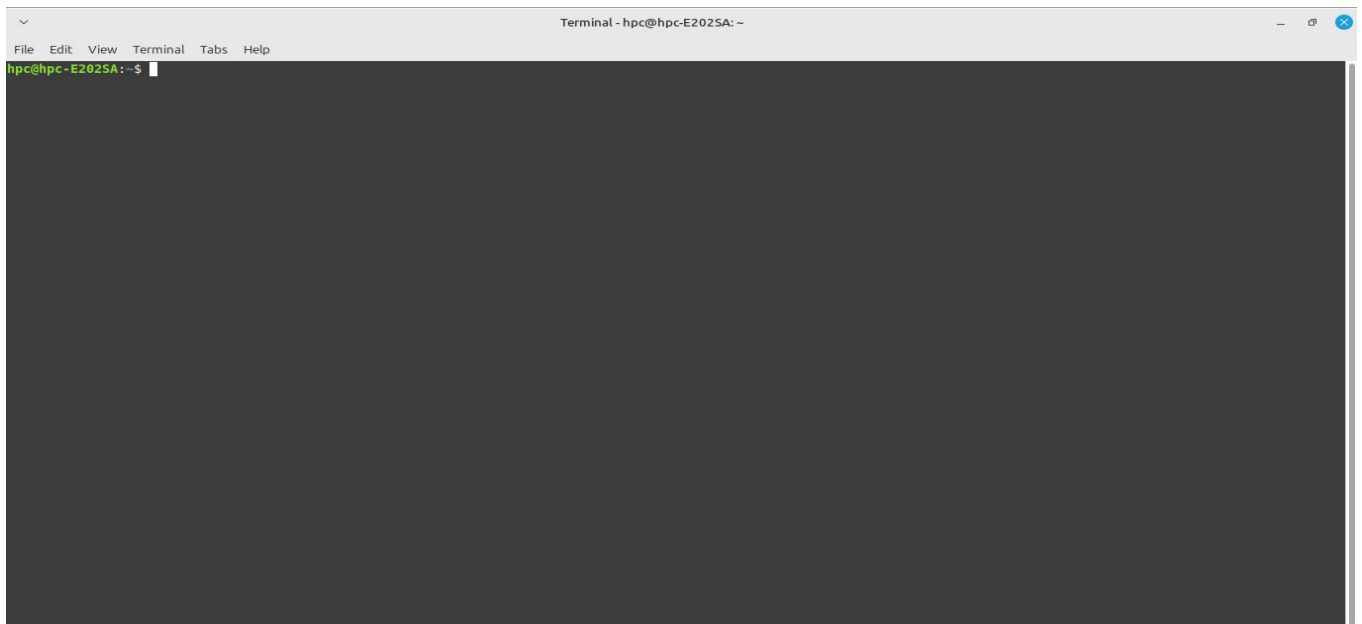
INDEX

Sl. No.	Name of the Experiment
1	Installation and Configuration of Linux OS (Ubuntu/Fedora)
2	Basic Linux Commands (ls, cd, mkdir, rm, cp, mv)
3	File Permissions and Ownership (chmod, chown, chgrp)
4	Creating and Editing Shell Scripts
5	Networking Commands (ping, ifconfig/ip, netstat)
6	Process Creation using fork(), exec(), and wait()
7	FCFS (First Come First Served) Scheduling Algorithm
8	Round Robin Scheduling Algorithm
9	Priority Scheduling (Preemptive/ Non-Preemptive)
10	Producer–Consumer Problem using Semaphores
11	Reader–Writer Problem using Mutex and Condition Variables
12	Deadlock Detection and Prevention Algorithms
13	Page Replacement Algorithms (FIFO, LRU)
14	File Operations using System Calls (open, read, write, close)
15	Mini Project / Project Evaluation

Experiment 1: Installation of Linux OS

Aim: To install and configure Linux Operating System.

Installation has been done successfully and final terminal screenshot has been uploaded..



Result : LINUX mint OS installed successfully .

Terminal name as follow :

hpc@hpc-E202SA

Experiment 2: Basic Linux Commands

Aim: To study basic Linux commands for file and directory manipulation

Commands Used:

- **pwd** : shows current directory
- **ls** : lists files
- **mkdir** : creates directory
- **cd** : changes directory
- **touch** : creates file
- **rm** : deletes file
- **rm -r folder_name** : deletes folder
- **cp** : copies file/folder
- **mv** : moves or renames file
- **cd ..** : moves to the parent (previous)directory

```
Terminal - hpc@hpc-E202SA: ~
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ pwd
/home/hpc
hpc@hpc-E202SA:~$ ls
Assignment Assignment2 Desktop Documents Downloads Music Pictures Public SQL sri Templates Videos
hpc@hpc-E202SA:~$ mkdir os
hpc@hpc-E202SA:~$ cd os
hpc@hpc-E202SA:~/os$ touch first.txt
hpc@hpc-E202SA:~/os$ ls
first.txt
hpc@hpc-E202SA:~/os$ rm first.txt
hpc@hpc-E202SA:~/os$ ls
hpc@hpc-E202SA:~/os$ touch fif.txt
hpc@hpc-E202SA:~/os$ ls
fif.txt
hpc@hpc-E202SA:~/os$ cp fif.txt sis.txt
hpc@hpc-E202SA:~/os$ ls
fif.txt sis.txt
hpc@hpc-E202SA:~/os$ mv fif.txt cic.txt
hpc@hpc-E202SA:~/os$ ls
cic.txt sis.txt
hpc@hpc-E202SA:~/os$ pwd
/home/hpc/os
hpc@hpc-E202SA:~/os$ cd ..
hpc@hpc-E202SA:~$ pwd
/home/hpc
hpc@hpc-E202SA:~$ rm -r os
hpc@hpc-E202SA:~$ ls
Assignment Assignment2 Desktop Documents Downloads Music Pictures Public SQL sri Templates Videos
hpc@hpc-E202SA:~$
```

Result : File and directory operations executed successfully.

Experiment 3: File Permissions and Ownership

Aim: To understand Linux file permissions.

Commands:

- **ls -l** : view file permissions and ownership
- **chmod** : change file permissions
- **chown** : change file owner (and group)
- **chgrp** : change group ownership
- **adduser** : creates a new user interactively with home directory and password
- **addgroup** : creates a new group

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch fif.txt
hpc@hpc-E202SA:~/sri/os$ ls
fif.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rw-rw-r-- 1 hpc hpc 0 Jan 20 00:17 fif.txt
hpc@hpc-E202SA:~/sri/os$ chmod 777 fif.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:17 fif.txt
hpc@hpc-E202SA:~/sri/os$ touch sis.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:17 fif.txt
-rw-rw-r-- 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$ chmod 755 sis.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:17 fif.txt
-rwxr-xr-x 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$ chmod u-x,g+w sis.txt
hpc@hpc-E202SA:~/sri/os$ chmod u-wx,g-w,o-r fif.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-r--r-x-wx 1 hpc hpc 0 Jan 20 00:17 fif.txt
-rw-rwxr-x 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$ chmod 777 fif.txt
hpc@hpc-E202SA:~/sri/os$ chmod 777 sis.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:17 fif.txt
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$
```

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ sudo adduser srishanth
[sudo] password for hpc:
info: Adding user `srishanth' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `srishanth' (1001) ...
info: Adding new user `srishanth' (1001) with group `srishanth (1001)' ...
info: Creating home directory `/home/srishanth' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for srishanth
Enter the new value, or press ENTER for the default
  Full Name []: srishanth
  Room Number []: 47
  Work Phone []: 123456
  Home Phone []: 12345677
  Other []: 12
Is the information correct? [Y/n] y
info: Adding new user `srishanth' to supplemental / extra groups `users' ...
info: Adding user `srishanth' to group `users' ...
hpc@hpc-E202SA:~$ sudo addgroup hello
info: Selecting GID from range 1000 to 59999 ...
info: Adding group `hello' (GID 1003) ...
```

```
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ sudo chown srishanth:hello fif.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 srishanth hello 0 Jan 20 00:17 fif.txt
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$ sudo chown hpc:hpc fif.txt
hpc@hpc-E202SA:~/sri/os$ ls -l
total 0
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:17 fif.txt
-rwxrwxrwx 1 hpc hpc 0 Jan 20 00:19 sis.txt
hpc@hpc-E202SA:~/sri/os$ cd ..
```


Experiment 4: Shell Scripting

Aim: To create and edit shell scripts for automate tasks

Commands :

- **.sh (extension)** : shell script file
- **touch file_name** : create a shell script file
- **nano file_name** : edit the script (can also use **vi**)
- **cat file_name** : display script content
- **./file_name** : run the script

A terminal window titled "Terminal - hpc@hpc-E202SA: ~" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows a series of commands and their outputs. The user navigates to the /sri directory, then to the os subdirectory. They create a file named script.sh using the touch command, then edit it with nano. The content of script.sh is "My name is SRI" and "Studying AMITY UNIVERSITY BENGALURU...". They then create another file script2.sh, make it executable with chmod, and run it with ./script2.sh, which outputs "HELLO AMITIANS". Finally, they navigate back to the parent directory and then to the sri directory.

```
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch script.sh
hpc@hpc-E202SA:~/sri/os$ nano script.sh
hpc@hpc-E202SA:~/sri/os$ cat script.sh
My name is SRI
Studying AMITY UNIVERSITY BENGALURU...
hpc@hpc-E202SA:~/sri/os$ touch script2.sh
hpc@hpc-E202SA:~/sri/os$ nano script2.sh
hpc@hpc-E202SA:~/sri/os$ chmod +x script2.sh
hpc@hpc-E202SA:~/sri/os$ ./script2.sh
HELLO AMITIANS
hpc@hpc-E202SA:~/sri/os$ cd ..
hpc@hpc-E202SA:~/sri$ cd ..
hpc@hpc-E202SA:~$
```


Experiment 5: Networking Commands

Aim: To study basic networking commands.

Commands :

- **ifconfig** : display network interface and IP address
- **ping website_name** : verify internet connectivity (**ctrl+c** to stop the ping)
- **ip route** : display routing table
- **netstat** : display active network connections

```
Terminal - hpc@hpc-E202SA:~
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ ping google.com
PING google.com (142.250.206.110) 56(84) bytes of data:
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=1 ttl=117 time=254 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=2 ttl=117 time=483 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=3 ttl=117 time=202 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=4 ttl=117 time=139 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=5 ttl=117 time=355 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=6 ttl=117 time=48.1 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=7 ttl=117 time=34.5 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=8 ttl=117 time=44.5 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=9 ttl=117 time=277 ms
64 bytes from lcboma-az-in-f14.1e100.net (142.250.206.110): icmp_seq=10 ttl=117 time=150 ms
^C
--- google.com ping statistics ---
11 packets transmitted, 10 received, 9.09091% packet loss, time 10016ms
rtt min/avg/max/mdev = 34.513/198.739/483.441/139.343 ms
hpc@hpc-E202SA:~$ ifconfig
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 133 bytes 11558 (11.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 133 bytes 11558 (11.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp1s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.106.113 netmask 255.255.255.0 broadcast 192.168.106.255
    inet6 fe80::4c85:f732:269e:fe60 prefixlen 64 scopeid 0x20<link>
    ether 60:45:cb:b4:d8:74 txqueuelen 1000 (Ethernet)
    RX packets 139 bytes 22671 (22.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 156 bytes 23176 (23.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

hpc@hpc-E202SA:~$ ip route
default via 192.168.106.112 dev wlp1s0 proto dhcp src 192.168.106.113 metric 600
192.168.106.0/24 dev wlp1s0 proto kernel scope link src 192.168.106.113 metric 600
hpc@hpc-E202SA:~$
```

```
Terminal - hpc@hpc-E2025A: ~
File Edit View Terminal Tabs Help

hpc@hpc-E2025A:~$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 hpc-E2025A:bootpc      _gateway:bootps        ESTABLISHED

Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags     Type       State      I-Node  Path
unix   3      [ ]      STREAM    CONNECTED  14453    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  14664
unix   3      [ ]      STREAM    CONNECTED  13202
unix   3      [ ]      STREAM    CONNECTED  11198    /run/user/1000/bus
unix   2      [ ]      DGRAM     CONNECTED  8826
unix   3      [ ]      STREAM    CONNECTED  9270
unix   3      [ ]      STREAM    CONNECTED  12524    /run/systemd/journal/stdout
unix   3      [ ]      STREAM    CONNECTED  9740
unix   2      [ ]      DGRAM     CONNECTED  8755
unix   3      [ ]      STREAM    CONNECTED  13101    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  8756
unix   3      [ ]      STREAM    CONNECTED  6114    /run/systemd/journal/stdout
unix   3      [ ]      STREAM    CONNECTED  14452
unix   3      [ ]      STREAM    CONNECTED  13158
unix   3      [ ]      STREAM    CONNECTED  12411    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  8820    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  9779    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  13163    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  13150    /run/systemd/journal/stdout
unix   2      [ ]      DGRAM     CONNECTED  12401
unix   3      [ ]      STREAM    CONNECTED  12537    /run/systemd/journal/stdout
unix   2      [ ]      DGRAM     CONNECTED  9772
unix   3      [ ]      STREAM    CONNECTED  14385
unix   3      [ ]      STREAM    CONNECTED  13415
unix   3      [ ]      STREAM    CONNECTED  13118    /run/systemd/journal/stdout
unix   2      [ ]      DGRAM     CONNECTED  11196
unix   2      [ ]      DGRAM     CONNECTED  8789
unix   3      [ ]      STREAM    CONNECTED  13229    @/tmp/.X11-unix/X0
unix   3      [ ]      STREAM    CONNECTED  13105    /run/user/1000/at-spi-bus_0
unix   3      [ ]      STREAM    CONNECTED  13376
unix   3      [ ]      STREAM    CONNECTED  9862    /run/systemd/journal/stdout
unix   3      [ ]      STREAM    CONNECTED  8790
unix   3      [ ]      STREAM    CONNECTED  14655
```

there are multiple connection details in between (in between are skipped and kept beginning and final screenshot)

```
Terminal - hpc@hpc-E2025A: ~
File Edit View Terminal Tabs Help

unix   2      [ ]      DGRAM     CONNECTED  10533
unix   5      [ ]      DGRAM     CONNECTED  974      /run/systemd/notify
unix   3      [ ]      STREAM    CONNECTED  13922    /run/user/1000/pipewire-0
unix   3      [ ]      STREAM    CONNECTED  13280
unix   3      [ ]      STREAM    CONNECTED  13627    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  12673    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  14027    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  13305    @/tmp/.X11-unix/X0
unix   3      [ ]      STREAM    CONNECTED  10389    /run/systemd/journal/stdout
unix   3      [ ]      STREAM    CONNECTED  11363    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  14035
unix   3      [ ]      STREAM    CONNECTED  13629    /run/user/1000/at-spi-bus_0
unix   3      [ ]      STREAM    CONNECTED  13372    /run/systemd/journal/stdout
unix   2      [ ]      DGRAM     CONNECTED  10212    /run/wpa_supplicant/wlplso
unix   3      [ ]      STREAM    CONNECTED  10347    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  8943
unix   3      [ ]      STREAM    CONNECTED  14041
unix   3      [ ]      STREAM    CONNECTED  14036
unix   3      [ ]      STREAM    CONNECTED  13049    @/tmp/.X11-unix/X0
unix   3      [ ]      STREAM    CONNECTED  12474
unix   2      [ ]      DGRAM     CONNECTED  997      /run/systemd/journal/syslog
unix   3      [ ]      STREAM    CONNECTED  14043    /run/user/1000/bus
unix   3      [ ]      STREAM    CONNECTED  11473    @/tmp/.X11-unix/X0
unix   3      [ ]      STREAM    CONNECTED  13628    /run/user/1000/at-spi-bus_0
unix   3      [ ]      STREAM    CONNECTED  14078    /run/systemd/journal/stdout
unix   3      [ ]      STREAM    CONNECTED  12505
unix   16     [ ]      DGRAM     CONNECTED  1003    /run/systemd/journal/dev-log
unix   3      [ ]      STREAM    CONNECTED  13285    /run/user/1000/at-spi-bus_0
unix   3      [ ]      STREAM    CONNECTED  10341    /run/dbus/system_bus_socket
unix   8      [ ]      DGRAM     CONNECTED  1005    /run/systemd/journal/socket
unix   3      [ ]      STREAM    CONNECTED  11361    /run/dbus/system_bus_socket
unix   3      [ ]      STREAM    CONNECTED  12407    @98813c5e9759300f/bus/systemd/bus-api-user
unix   2      [ ]      STREAM    CONNECTED  14380    @printer-applet-lock-user-hpc
unix   3      [ ]      STREAM    CONNECTED  8328    @eda5653fca0bac20/bus/systemd-resolve/bus-api-resolve
unix   3      [ ]      STREAM    CONNECTED  8327    @9088c297c17d827/bus/systemd-timesyn/bus-api-timesync
unix   3      [ ]      STREAM    CONNECTED  9624    @b6846de4da96309d/bus/systemd-logind/system
unix   3      [ ]      STREAM    CONNECTED  12255    @483b697bc266c26d/bus/systemd/bus-system
unix   3      [ ]      STREAM    CONNECTED  9780    @9c615e6bd878f27f/bus/systemd/bus-api-system

hpc@hpc-E2025A:~$
```

Experiment 6: Process Creation using System Calls

Aim: To create ,execute and terminate processes.

System Calls: fork(), exec(), wait().

Code :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
        printf("Fork failed\n");
    }
}
```

```
else if (pid == 0)
{
    printf("Child Process\n");
    execlp("ls", "ls", NULL);
}
else
{
    wait(NULL);
    printf("Parent Process\n");
}
return 0;
}
```



```
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch process.c
hpc@hpc-E202SA:~/sri/os$ nano process.c
hpc@hpc-E202SA:~/sri/os$ gcc process.c -o process
hpc@hpc-E202SA:~/sri/os$ ./process
Child Process
fif.txt process process.c script2.sh script.sh sis.txt
Parent Process
hpc@hpc-E202SA:~/sri/os$ cd ..
hpc@hpc-E202SA:~/sri$ cd ..
hpc@hpc-E202SA:~$
```

Experiment 7: FCFS Scheduling

Algorithm

Aim: To implement FCFS scheduling.

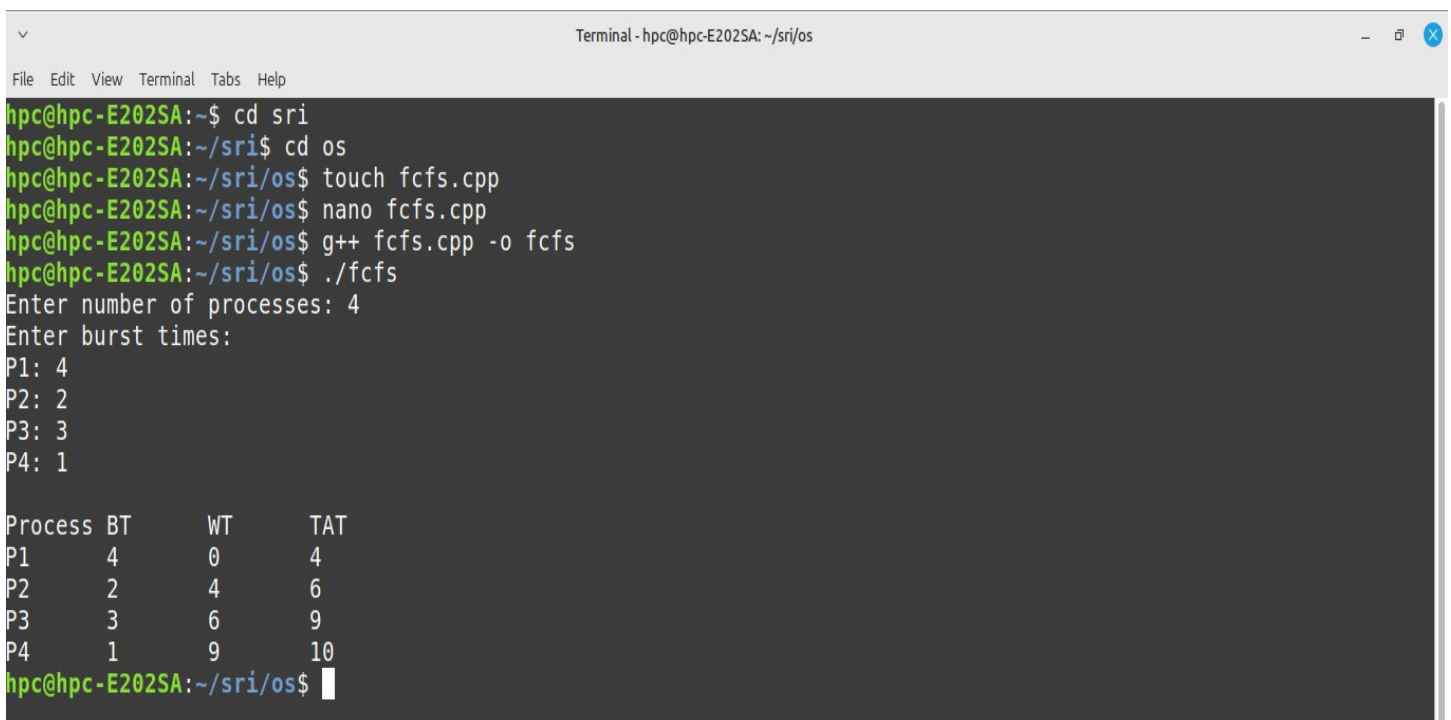
C++ program for FCFS :

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int bt[n], wt[n], tat[n];
    cout << "Enter burst times:\n";
    for (int i = 0; i < n; i++) {
        cout << "P" << i+1 << ": ";
        cin >> bt[i]; }
}
```

```

wt[0] = 0;
for (int i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1]; }
for (int i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i]; }
cout << "\nProcess\tBT\tWT\tTAT\n";
for (int i = 0; i < n; i++) {
    cout << "P" << i+1 << "\t" << bt[i] <<
"\t" << wt[i] << "\t" << tat[i] << endl; }
return 0;
}

```



```

Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch fcfs.cpp
hpc@hpc-E202SA:~/sri/os$ nano fcfs.cpp
hpc@hpc-E202SA:~/sri/os$ g++ fcfs.cpp -o fcfs
hpc@hpc-E202SA:~/sri/os$ ./fcfs
Enter number of processes: 4
Enter burst times:
P1: 4
P2: 2
P3: 3
P4: 1

Process BT      WT      TAT
P1      4        0        4
P2      2        4        6
P3      3        6        9
P4      1        9       10
hpc@hpc-E202SA:~/sri/os$

```

Experiment 8: Round Robin Scheduling

Aim: To implement Round Robin algorithm (Time quantum based scheduling.)

c++ code for Round robin :

```
#include <iostream>

using namespace std;

int main()
{
    int n, tq;
    cout << "Enter number of processes: ";
    cin >> n;
    int bt[n], rem_bt[n], wt[n], tat[n];
    cout << "Enter burst times:\n";
    for (int i = 0; i < n; i++)
    {
        cout << "P" << i + 1 << ": ";
        cin >> bt[i];
```



```
    rem_bt[i] = bt[i];
    wt[i] = 0;
}
cout << "Enter time quantum: ";
cin >> tq;
int time = 0;
bool done;
do
{
    done = true;
    for (int i = 0; i < n; i++)
    {
        if (rem_bt[i] > 0)
        {
            done = false;
            if (rem_bt[i] > tq)
            {
```

```
        time += tq;
        rem_bt[i] -= tq;
    }
    else
    {
        time += rem_bt[i];
        wt[i] = time - bt[i];
        rem_bt[i] = 0;
    }
}

}

} while (!done);
for (int i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
}

cout << "\nProcess\tBT\tWT\tTAT\n";
```

```

for (int i = 0; i < n; i++)
{
    cout << "P" << i + 1 << "\t"
        << bt[i] << "\t"
        << wt[i] << "\t"
        << tat[i] << endl;
}
return 0;
}

```

Output :

```

Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch rr.cpp
hpc@hpc-E202SA:~/sri/os$ nano rr.cpp
hpc@hpc-E202SA:~/sri/os$ g++ rr.cpp -o rr
hpc@hpc-E202SA:~/sri/os$ ./rr
Enter number of processes: 4
Enter burst times:
P1: 2
P2: 5
P3: 1
P4: 4
Enter time quantum: 2

Process BT    WT    TAT
P1      2      0      2
P2      5      7     12
P3      1      4      5
P4      4      7     11
hpc@hpc-E202SA:~/sri/os$

```

Experiment 9: Priority Scheduling

Aim: To simulate priority scheduling (preemptive/non-preemptive).

C++ code for priority scheduling :

```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter number of processes: ";
    cin >> n;
    int bt[n], pr[n], wt[n], tat[n];
    int temp;
    cout << "Enter burst time and priority:\n";
    for (int i = 0; i < n; i++)
    {
        cout << "P" << i + 1 << " Burst Time: ";
```

```
cin >> bt[i];
```

```
cout << "P" << i + 1 << " Priority: ";
```

```
cin >> pr[i];
```

```
}
```

```
// Sorting based on priority (lower value  
= higher priority)
```

```
for (int i = 0; i < n - 1; i++)
```

```
{
```

```
    for (int j = i + 1; j < n; j++)
```

```
    {
```

```
        if (pr[i] > pr[j])
```

```
        {
```

```
            temp = pr[i];
```

```
            pr[i] = pr[j];
```

```
            pr[j] = temp;
```

```
            temp = bt[i];
```

```
            bt[i] = bt[j];
```

```
            bt[j] = temp;
```

```

        }
    }
}
wt[0] = 0;
for (int i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}
for (int i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
}
cout << "\nProcess\tBT\tPriority\tWT\tTAT\n";
for (int i = 0; i < n; i++)
{
    cout << "P" << i + 1 << "\t"
        << bt[i] << "\t"

```

```
<< pr[i] << "\t\t"
<< wt[i] << "\t"
<< tat[i] << endl;
}

return 0;
}
```

Output :

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch priority.cpp
hpc@hpc-E202SA:~/sri/os$ nano priority.cpp
hpc@hpc-E202SA:~/sri/os$ g++ priority.cpp -o priority
hpc@hpc-E202SA:~/sri/os$ ./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 4
P1 Priority: 1
P2 Burst Time: 3
P2 Priority: 2
P3 Burst Time: 5
P3 Priority: 4
P4 Burst Time: 2
P4 Priority: 3
P5 Burst Time: 1
P5 Priority: 5

Process BT      Priority      WT      TAT
P1      4        1           0        4
P2      3        2           4        7
P3      2        3           7        9
P4      5        4           9       14
P5      1        5          14       15
hpc@hpc-E202SA:~/sri/os$
```


Experiment 10: Producer Consumer Problem

Aim: Implementing classical synchronization problem using Semaphores.

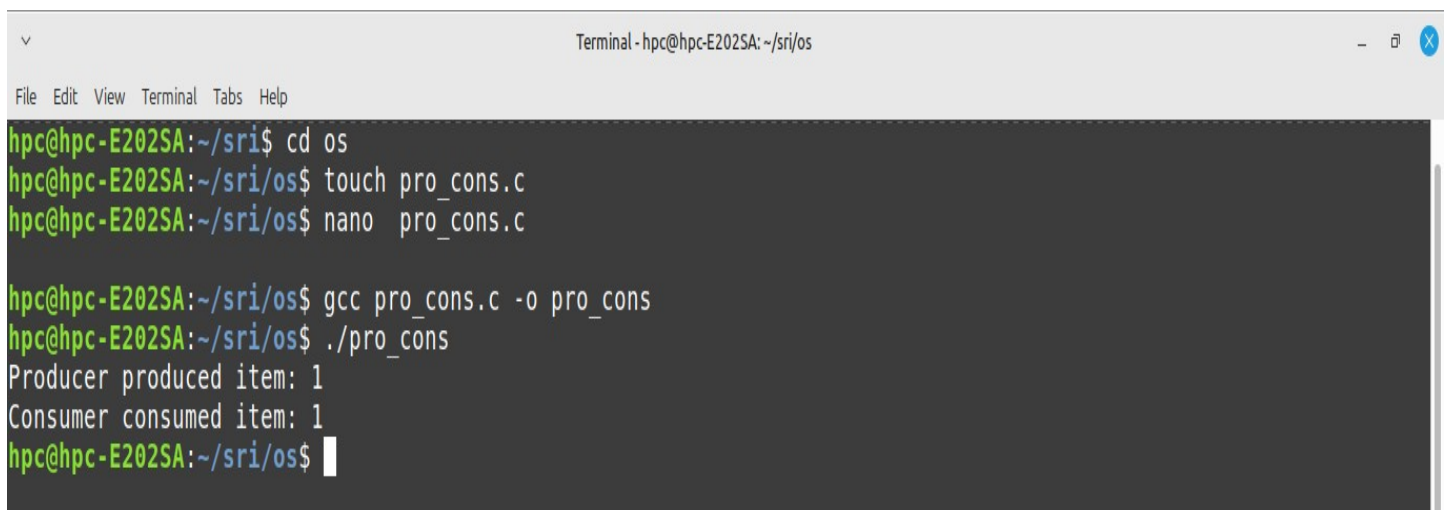
C program code :

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t empty, full;
int buffer = 0;
void *producer(void *arg)
{
    sem_wait(&empty);
    buffer++;
    printf("Producer produced item: %d\n",
buffer);
```

```
    sem_post(&full);
    sleep(1);
}
void *consumer(void *arg)
{
    sem_wait(&full);
    printf("Consumer consumed item: %d\n", buffer);
    buffer--;
    sem_post(&empty);
    sleep(1);
}
int main()
{
    pthread_t p, c;
    sem_init(&empty, 0, 1);
    sem_init(&full, 0, 0);
```

```
pthread_create(&p, NULL, producer,  
NULL);  
  
pthread_create(&c, NULL, consumer,  
NULL);  
  
pthread_join(p, NULL);  
pthread_join(c, NULL);  
sem_destroy(&empty);  
sem_destroy(&full);  
return 0;  
}
```

Output :

A terminal window titled "Terminal - hpc@hpc-E202SA: ~/sri/os" with a menu bar (File, Edit, View, Terminal, Tabs, Help). The terminal shows the following commands and output:

```
hpc@hpc-E202SA:~/sri$ cd os  
hpc@hpc-E202SA:~/sri/os$ touch pro_cons.c  
hpc@hpc-E202SA:~/sri/os$ nano pro_cons.c  
  
hpc@hpc-E202SA:~/sri/os$ gcc pro_cons.c -o pro_cons  
hpc@hpc-E202SA:~/sri/os$ ./pro_cons  
Producer produced item: 1  
Consumer consumed item: 1  
hpc@hpc-E202SA:~/sri/os$
```

Experiment 11: Reader Writer Problem

Aim: To implement Reader-Writer problem using **Mutex and condition variables**.

C code :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t mutex;
int readcount = 0;
void *reader(void *arg)
{
    pthread_mutex_lock(&mutex);
    readcount++;
    printf("Reader is reading, count = %d\n", readcount);
    pthread_mutex_unlock(&mutex);
}
```

```
    sleep(1);
    pthread_mutex_lock(&mutex);
    readcount--;
    pthread_mutex_unlock(&mutex);
}

void *writer(void *arg)
{
    pthread_mutex_lock(&mutex);
    printf("Writer is writing\n");
    sleep(1);
    pthread_mutex_unlock(&mutex);
}

int main()
{
    pthread_t r1, r2, w1;
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&r1, NULL, reader,
NULL);
```

```
    pthread_create(&w1, NULL, writer,
NULL);

    pthread_create(&r2, NULL, reader,
NULL);

    pthread_join(r1, NULL);
    pthread_join(w1, NULL);
    pthread_join(r2, NULL);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Output :

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~/sri/os$ nano read_writer.c
hpc@hpc-E202SA:~/sri/os$ gcc read_writer.c -o read_writer
hpc@hpc-E202SA:~/sri/os$ ./read_writer
Reader is reading, count = 1
Writer is writing
Reader is reading, count = 2
hpc@hpc-E202SA:~/sri/os$
```

Experiment 12: Deadlock Simulation

Aim: To simulate deadlock detection and prevention .

C code for Deadlock detection :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j;
```

```
    int alloc[10][10], req[10][10], avail[10];
```

```
    int finish[10] = {0}, work[10];
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        for (j = 0; j < m; j++)
```

```
scanf("%d", &alloc[i][j]);
printf("Enter Request Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &req[i][j]);
printf("Enter Available Resources:\n");
for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);
for (j = 0; j < m; j++)
    work[j] = avail[j];
int found;
do
{
    found = 0;
    for (i = 0; i < n; i++)
    {
        if (!finish[i])
```



```
{  
    int can_execute = 1;  
    for (j = 0; j < m; j++)  
    {  
        if (req[i][j] > work[j])  
        {  
            can_execute = 0;  
            break;  
        }  
    }  
    if (can_execute)  
    {  
        for (j = 0; j < m; j++)  
            work[j] += alloc[i][j];  
        finish[i] = 1;  
        found = 1;  
    }  
}
```

```
        }
    }
} while (found);
int deadlock = 0;
for (i = 0; i < n; i++)
{
    if (!finish[i])
    {
        deadlock = 1;
        printf("Process P%d is in
deadlock\n", i);
    }
}
if (!deadlock)
    printf("No deadlock detected\n");
return 0;
}
```

Output :

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~/sri/os$ touch deadlockDET.c
hpc@hpc-E202SA:~/sri/os$ nano deadlockDET.c
hpc@hpc-E202SA:~/sri/os$ gcc deadlockDET.c -o deadlockDET
hpc@hpc-E202SA:~/sri/os$ ./deadlockDET
Enter number of processes: 3
Enter number of resources: 3
Enter Allocation Matrix:
5 4 3 4 1 0
2 4 8
Enter Request Matrix:
7 8 9
3 5 8
1 9 0
Enter Available Resources:
3
2
4
Process P0 is in deadlock
Process P1 is in deadlock
Process P2 is in deadlock
hpc@hpc-E202SA:~/sri/os$
```

C code for Deadlock Prevention (Banker's Algorithm) :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, m, i, j;
```

```
    int alloc[10][10], max[10][10], need[10][10];
```

```
    int avail[10], finish[10] = {0}, safeSeq[10];
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resources: ");
scanf("%d", &m);
printf("Enter Allocation Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &alloc[i][j]);
printf("Enter Maximum Matrix:\n");
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        scanf("%d", &max[i][j]);
printf("Enter Available Resources:\n");
for (j = 0; j < m; j++)
    scanf("%d", &avail[j]);
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
```

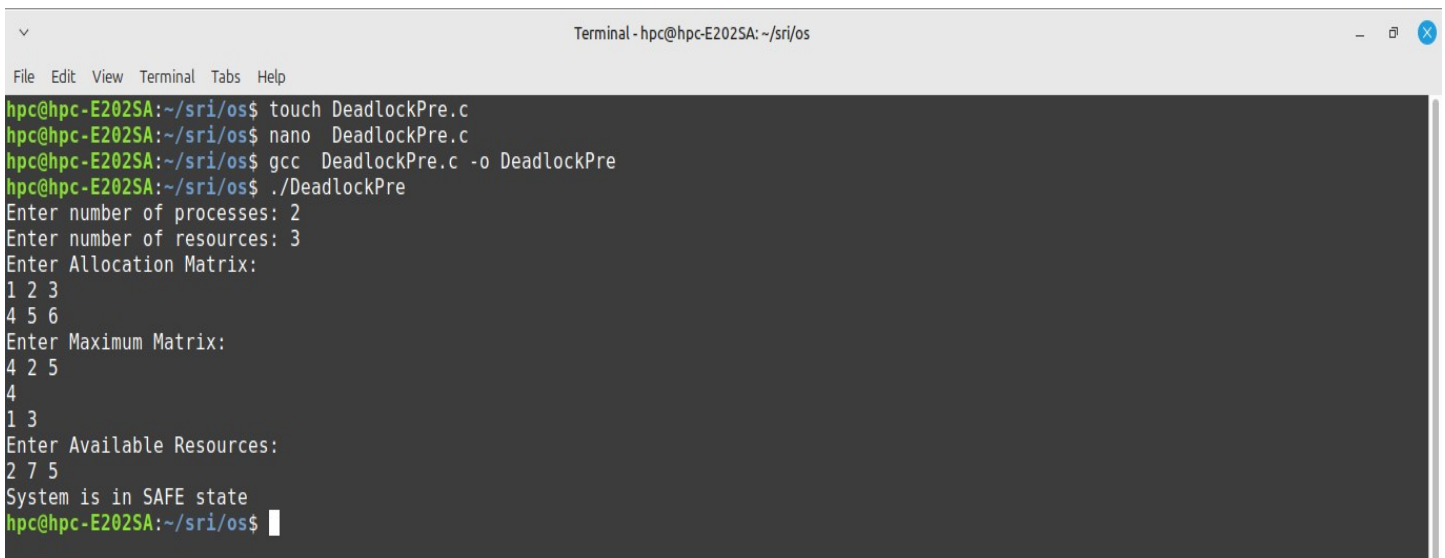
```
need[i][j] = max[i][j] - alloc[i][j];  
int count = 0;  
while (count < n)  
{  
    int found = 0;  
    for (i = 0; i < n; i++)  
    {  
        if (!finish[i])  
        {  
            int can_run = 1;  
            for (j = 0; j < m; j++)  
            {  
                if (need[i][j] > avail[j])  
                {  
                    can_run = 0;  
                    break;  
                }  
            }  
        }  
    }  
}
```

```
    }
    if (can_run)
    {
        for (j = 0; j < m; j++)
            avail[j] += alloc[i][j];
        safeSeq[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

}
if (!found)
{
    printf("System is in UNSAFE state
(Deadlock possible)\n");
    return 0;
}
}
```

```
    printf("System is in SAFE state\nSafe
Sequence: ");
    for (i = 0; i < n; i++)
        printf("P%d ", safeSeq[i]);
    return 0;
}
```

Output :

A terminal window titled "Terminal - hpc@hpc-E202SA: ~/sri/os" showing the execution of a program. The user enters commands to create, edit, compile, and run a file named DeadlockPre.c. The program prompts for the number of processes (2), number of resources (3), an allocation matrix, a maximum matrix, and available resources. It then outputs "System is in SAFE state".

```
hpc@hpc-E202SA:~/sri/os$ touch DeadlockPre.c
hpc@hpc-E202SA:~/sri/os$ nano DeadlockPre.c
hpc@hpc-E202SA:~/sri/os$ gcc DeadlockPre.c -o DeadlockPre
hpc@hpc-E202SA:~/sri/os$ ./DeadlockPre
Enter number of processes: 2
Enter number of resources: 3
Enter Allocation Matrix:
1 2 3
4 5 6
Enter Maximum Matrix:
4 2 5
4
1 3
Enter Available Resources:
2 7 5
System is in SAFE state
hpc@hpc-E202SA:~/sri/os$
```

Experiment 13: Page Replacement Algorithms

Aim: To implement Page replacement by FIFO and LRU simulation.

C code for FIFO Page replacement :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int pages[20], frames[10];
```

```
    int n, f, i, j, k = 0;
```

```
    int faults = 0, found;
```

```
    printf("Enter number of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &f);
```



```
for (i = 0; i < f; i++)
    frames[i] = -1;
printf("\nFIFO Page Replacement\n");
for (i = 0; i < n; i++)
{
    found = 0;
    for (j = 0; j < f; j++)
    {
        if (frames[j] == pages[i])
        {
            found = 1;
            break;
        }
    }
    if (!found)
    {
        frames[k] = pages[i];
    }
}
```

```

        k = (k + 1) % f;
        faults++;
    }
    for (j = 0; j < f; j++)
        printf("%d ", frames[j]);
    printf("\n");
}

printf("Total Page Faults = %d\n",
faults);

return 0;
}

```

Output :

```

Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch FIFOpr.c
hpc@hpc-E202SA:~/sri/os$ nano FIFOpr.c
hpc@hpc-E202SA:~/sri/os$ gcc FIFOpr.c -o FIFOpr
hpc@hpc-E202SA:~/sri/os$ ./FIFOpr
Enter number of pages: 3
Enter page reference string:
2
3
2
Enter number of frames: 3

FIFO Page Replacement
2 -1 -1
2 3 -1
2 3 -1
Total Page Faults = 2
hpc@hpc-E202SA:~/sri/os$

```

C code for LRU Page Replacement :

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int pages[20], frames[10], time[10];
```

```
    int n, f, i, j, pos, faults = 0, found, count  
= 0;
```

```
    printf("Enter number of pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter page reference string:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter number of frames: ");
```

```
    scanf("%d", &f);
```

```
    for (i = 0; i < f; i++)
```

```
{
```

```
        frames[i] = -1;
```

```
        time[i] = 0;
```

```
}  
printf("\nLRU Page Replacement\n");  
for (i = 0; i < n; i++)  
{  
    found = 0;  
    for (j = 0; j < f; j++)  
    {  
        if (frames[j] == pages[i])  
        {  
            found = 1;  
            time[j] = ++count;  
            break;  
        }  
    }  
    if (!found)  
    {  
        pos = 0;
```

```
        for (j = 1; j < f; j++)
            if (time[j] < time[pos])
                pos = j;
        frames[pos] = pages[i];
        time[pos] = ++count;
        faults++;
    }
    for (j = 0; j < f; j++)
        printf("%d ", frames[j]);
    printf("\n");
}

printf("Total Page Faults = %d\n",
faults);

return 0;
}
```

Output :

```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch LRUp.c
hpc@hpc-E202SA:~/sri/os$ nano LRUp.c
hpc@hpc-E202SA:~/sri/os$ gcc LRUp.c -o LRUp
./hpc@hpc-E202SA:~/sri/os$ ./LRUp
Enter number of pages: 6
Enter page reference string:
6
3
7
1
3
8
Enter number of frames: 3

LRU Page Replacement
6 -1 -1
6 3 -1
6 3 7
1 3 7
1 3 7
1 3 8
Total Page Faults = 5
hpc@hpc-E202SA:~/sri/os$
```

Simple terms :

FIFO : replaces page that came first.

LRU : replaces page not used recently.

PAGE FAULT : page not found in memory.

Experiment 14: File Operations using System Calls

Aim: To perform basic file operations using System Calls (open read, write, close) and demonstrating directory traversals.

C code for File operations using System calls :

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int fd;
```

```
    char writeData[] = "Operating System  
Lab File Operations\n";
```

```
    char readData[100];
```

```
    /* Open file */  
    fd = open("sample.txt", O_CREAT |  
O_RDWR, 0644);  
    /* Write to file */  
    write(fd, writeData, strlen(writeData));  
    /* Move file pointer to beginning */  
    lseek(fd, 0, SEEK_SET);  
    /* Read from file */  
    read(fd, readData, sizeof(readData));  
    printf("Data read from file:\n%s",  
readData);  
    /* Close file */  
    close(fd);  
    return 0;  
}
```

Output :


```
Terminal - hpc@hpc-E202SA: ~/sri/os
File Edit View Terminal Tabs Help
hpc@hpc-E202SA:~$ cd sri
hpc@hpc-E202SA:~/sri$ cd os
hpc@hpc-E202SA:~/sri/os$ touch FileOpSys.c
hpc@hpc-E202SA:~/sri/os$ nano FileOpSys.c
hpc@hpc-E202SA:~/sri/os$ gcc FileOpSys.c -o FileOpSys
hpc@hpc-E202SA:~/sri/os$ ./FileOpSys
Data read from file:
Operating System Lab File Operations
hpc@hpc-E202SA:~/sri/os$
```

C code for **Directory traversal** :

```
#include <stdio.h>
```

```
#include <dirent.h>
```

```
int main()
```

```
{
```

```
    DIR *d;
```

```
    struct dirent *dir;
```

```
    d = opendir(".");
```

```
    if (d == NULL)
```

```
    {
```

```
        printf("Cannot open directory\n");
```

```
        return 0;
```

```

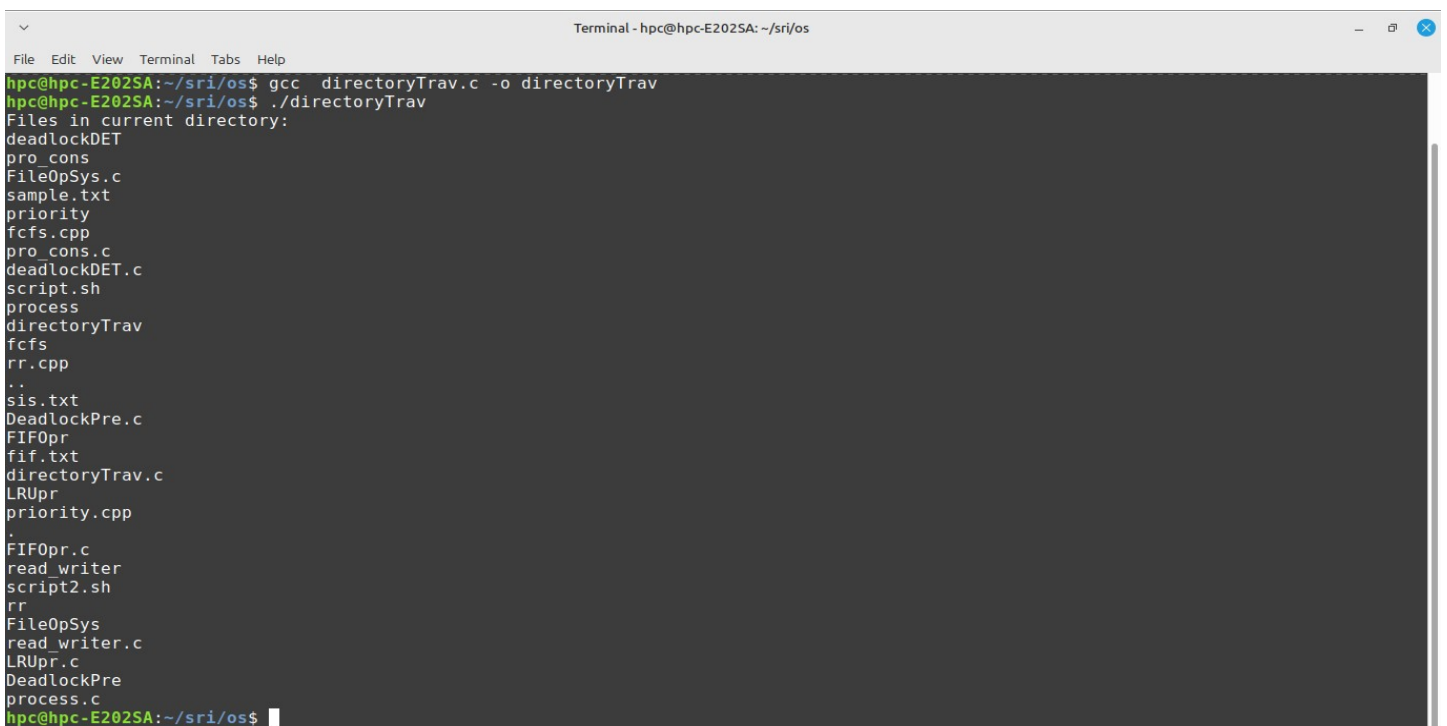
}

printf("Files in current directory:\n");
while ((dir = readdir(d)) != NULL)
{
    printf("%s\n", dir->d_name);
}

closedir(d);
return 0;
}

```

Output :



A terminal window titled "Terminal - hpc@hpc-E2025A: ~/sri/os" showing the compilation and execution of a C program. The user runs `gcc directoryTrav.c -o directoryTrav` and then `./directoryTrav`. The program outputs a list of files in the current directory.

```

hpc@hpc-E2025A:~/sri/os$ gcc directoryTrav.c -o directoryTrav
hpc@hpc-E2025A:~/sri/os$ ./directoryTrav
Files in current directory:
deadlockDET
pro_cons
FileOpSys.c
sample.txt
priority
fcfs.cpp
pro_cons.c
deadlockDET.c
script.sh
process
directoryTrav
fcfs
rr.cpp
..
sis.txt
DeadlockPre.c
FIFOpr
fif.txt
directoryTrav.c
LRUp
priority.cpp
.
FIFOpr.c
read_writer
script2.sh
rr
FileOpSys
read_writer.c
LRUp.c
DeadlockPre
process.c
hpc@hpc-E2025A:~/sri/os$

```

System calls used :

open() - opens or creates a file

read() - reads data from file

write() - writes data to file

close() - closes file

Directory traversals :

opendir() - opens directory

readdir() - reads directory entries

closedir() - closes directory

Result : File operations executed.

Conclusion:

Through this laboratory course, various operating system concepts and Linux commands were studied and implemented practically. This enhanced understanding of OS fundamentals and Linux internals.

Thank you SIR!

-----THE END-----